Unit 4

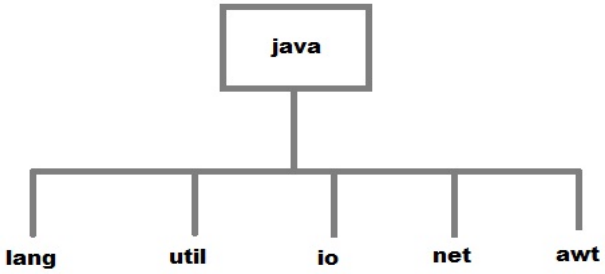
Packages and JAVA Library: Introduction, Defining Package, Importing Packages and Classes into Programs, Path and Class Path, Access Control, Packages in JAVA SE, java.lang Package and its Classes, Class Object, class Math, Wrapper Classes, Auto-boxing and Auto-unboxing.

Exception Handling: Introduction, Hierarchy of Standard Exception Classes, Keywords throws and throw, try, catch, and finally Blocks, Multiple Catch Clauses, Class Throwable, Unchecked Exceptions, Checked Exceptions, custom exceptions.

Java I/O and File: Java I/O API, standard I/O streams, types, Byte streams, Character streams, Scanner class, Files in Java: File, FileInputStream and FileOutputStream Classes

Packages and JAVA Library

- JAVA follows OOP
- Number of built in classes and interfaces are there
- For proper management of classes and interfaces, the related classes and interfaces are grouped into packages and sub-packages.
- The default package is java.
- A large number of the packages and sub-packages are contained in the package java.



Packages as Name Space

- A package provides a name space in which the classes and interfaces have unique names.
- Same names may be used in different packages without ambiguity.
- Similarly, the class name and interface also provide name spaces in which the names of variables and methods are unique.

Built-in Packages

Package Name	Description
java.lang	Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.) . This package is automatically imported.
java.io	Contains classes for supporting input / output operations.
java.util	Contains utility classes which implement data structures like List, Hash Table, Dictionary, etc and support for Date / Time operations. This package is also called as Collections.
java.applet	Contains classes for creating Applets.
java.awt	Contains classes for implementing the components of graphical user interface (like buttons, menus, etc.).
java.net	Contains classes for supporting networking operations.
javax.swing	This package helps to develop GUI like java.awt. The ‘x’ in javax represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, javax.swing is an extended package of java.awt.
java.sql	This package helps to connect to databases like Oracle/Sybase/Microsoft Access to perform different operations.

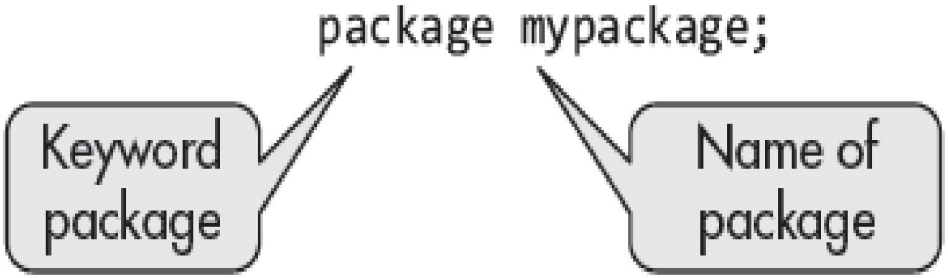
User defined packages

- One of the important features of the Java is that it allows a programmer to build his/her own packages and classes.
- A programmer can save his/her own packages and classes in a directory, and from there, it can be imported to the application program by using import keyword.
- To create a user defined package, in program the first instruction must be

  package packagename;

Defining a Package

- A package comprises a group of similar type of classes, interfaces, and sub-packages.
- It is defined with the keyword ***package*** followed by the name of the package and a semicolon



- **Package statement is the first statement in a program, No other code line can be placed above it expect the comments.**

Defining a Package

- Rules:
  - All the members of a package must have unique names, otherwise compile-type error will occur.
  - Packages also provide an access control mechanism.
  - A package that is not a sub-package of another package can have a simple name. Examples are java, javax, and javafx
  - Qualified name:
    - If a package contained another sub-packages we use qualified name.
    - Qualified name consists of names of pakages/sub-package separated by .(dot)

Defining a Package

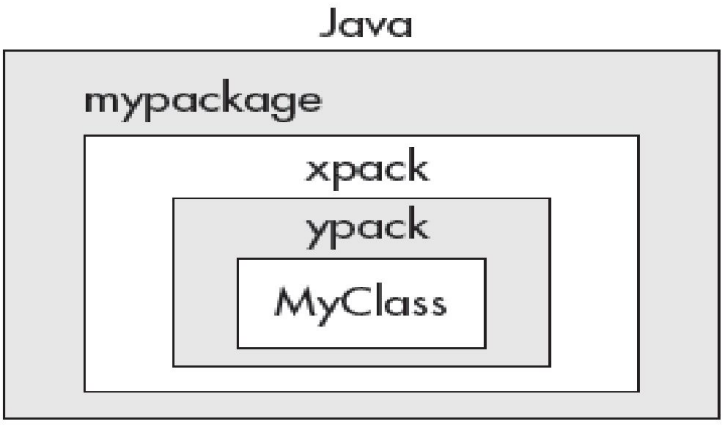


Fig. 10.1 Illustration of hierarchy of packages

```
java.mypackage.xpack.ypack.MyClass;
```

Different levels in packages are separated by a period (.)

package java.mypackage;// qualified name for mypackage

package java.mypackage.xpack; // qualified name for xpack

package java.mypackage.xpack.ypack;// qualified name for ypack

Importing Packages and Classes into Programs

- Most of the Java programs need to refer to one or more packages and classes in the source code.
- If a program needs several classes of a package, it is better to import the whole package

```
import java.awt.*;
```

Note the period (.) and star (*) after package name awt

- For a particular class may be included in a program by importing it through its fully qualified name

```
import java.packageP1.packageP2. packageP3. class_identifier;
```

Name of package

Name of sub packages

Name of class

```
import java.awt.Color;
```

Level 1 package

Package at level 2 and contains class Color

How to access package from another package?

There are three ways to access the package

- import package.*;
- import package.classname;
- fully qualified name.

Method	Syntax	When to use
Fully qualified name	pack.Addition a = new pack.Addition(10,15);	Using a class once, or avoiding name clashes
Import a single class	import pack.Addition;	Only a few classes needed
Import all classes	import pack.*;	Many classes from the same package

PATH and CLASSPATH

- PATH - is an environment variable is used to tell the location of built-in directories such as *javac, java, jar, javadoc, appletviewer ...*
- Path tells the system where to locate the JDK files that contain these commands.
- *CLASSPATH –*
    - The class path environment variable is used to specify the location of the user defined classes and packages.

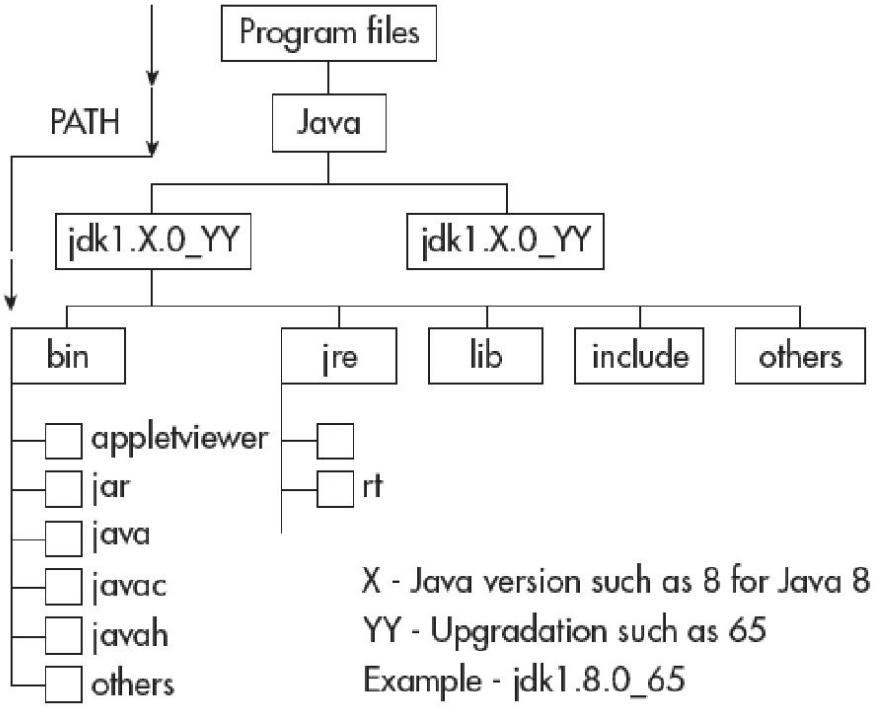


Fig. 10.4 Illustration of path to jdk-bin files

TO WORK WITH USER DEFINED PACKAGES – SETTING CLASSPATH

- First, by default, the Java run-time system uses the current working directory as its starting point.
- If your package is in a subdirectory of the current directory, it will be found.

- Second, you can specify a directory path or paths by setting the **CLASSPATH environmental variable.**
  - Ex: UseofPack.java is in E Drive and Package is in F drive

```
E:\>set CLASSPATH=F:\Courses\JAVA26-27\Unit4\Packages;.;
```

- Third, you can use the **–cp(classpath)** option with **java and javac** to specify the path to your classes.

```
E:\>javac -cp "F:\Courses\JAVA26-27\Unit4\Packages;.;" UseofPack.java

E:\>java -cp "F:\Courses\JAVA26-27\Unit4\Packages;.;" UseofPack
Sum :25
Difference :5
```

-

jar file(Java Archive)

- JAR (JAVA Archive) file is a file that contains compressed venison of class files, audio files, image files and directories.
- To create JAR file use jar command
  - jar cf outputfile_name.jar input_files
    - jar cf file_name
    - jar cvf file_name – create verbose
    - jar tf file_name - table view of file contents
    - jar xf file_name - extract file

How to use JAR files

- Go to the environment variables.
- Go to user **environment *variables*** and click on ***new*** button
- Set the classpath
  - Variable name CLASSPATH
  - Variable value

    F:\Courses\JAVA26-27\Unit4\Packages\PVPSIT.jar;.;

- Click OK and Close

Access Control

- The packages are containers of sub-packages, classes, and interfaces.
- The encapsulation of related classes in packages provides two benefits:

  (i) Name space

```
college.student
    └── Student

company.R_D
    └── Student
```

  (ii) Access protection - control which classes and members can be accessed from other packages.

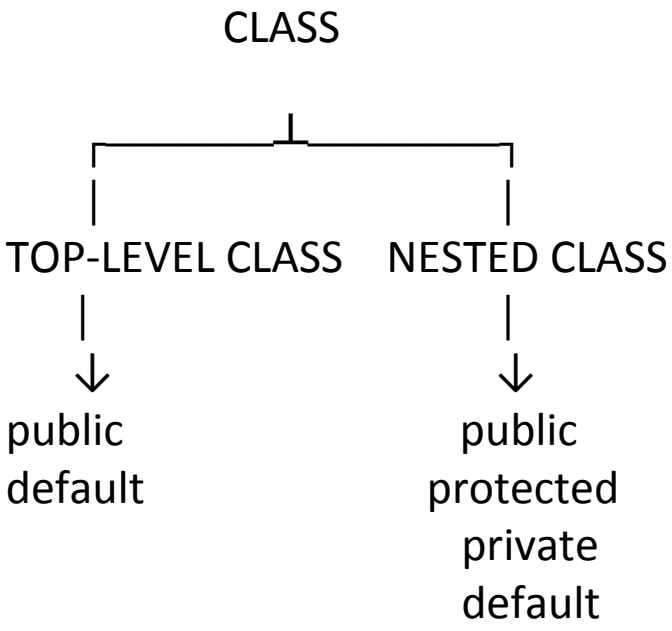
  Access is controlled using public, protected, default, and private.

Access specifiers for classes

- Generally, protected and private are used only in nested classes.

Table 10.1 Access specifiers for classes

Access specifier	*Example of code*	*Access permitted*
No access specifier	`class A {}`	Accessible to classes in same package
public	`public class B{}`	Accessible to all classes in all packages
protected	`public class A{` `protected class C {} }`	Used with nested classes; members of host (outer) class can access protected class because it is a member of the class
private	`public class A{` `private class C {} }`	Used with nested classes; members of host (outer) class can access private class because it is a member of the class



A top-level class can be only public or default, whereas a nested class can use all four access specifiers: public, protected, private, and default.

Access Control

Table 8.4 Access control

Class	*Access permitted (Yes/No)*			
	No specifier	*Public*	*Protected*	*Private*
Same class members	Yes	Yes	Yes	Yes
Subclass in same package	Yes	Yes	Yes	No
Any other class in same package	Yes	Yes	Yes	No
Subclass in different package	No	Yes	Yes	No
Any other class in different package	No	Yes	No	No

Control of access to class members

Table 10.2 Access specifiers and permissible access for class members and different packages

Access specifier	*Access*
No access specifier	Access permitted to any other class belonging to the same package
public	Access permitted to any class in any package
protected	Access permitted to its subclasses in any package and also to any class in the same package
private	Access permitted to members of the same class only; no access permitted to any code outside the class

java.lang Package and its classes

- java.lang package is imported into every Java program by default.
- This package contains classes and interfaces that are essential for the design of Java programming language and are as follows:
  - Object class is the root of all classes
  - Primitive data types
  - Classes access system resources and other low level entities
  - Math class – for mathematical operations
  - Class Throwable – for errors and exceptions
  - Thread class – to implement threads
  - To work with strings such as String, StringBuffer

- **Boolean** − The Boolean class wraps a value of the primitive type boolean in an object.
- **Byte** − The Byte class class wraps a value of primitive type byte in an object.
- **Character** − The Character class offers a number of useful class (i.e., static) methods for manipulating characters.
- **Class** − The Class class instance represent classes and interfaces in a running Java application. It has no public constructor.
- **String** − Strings, which are widely used in Java programming, are a sequence of characters. In Java programming language, strings are treated as objects. The Java platform provides the String class to create and manipulate strings.
- **StringBuffer** − The StringBuffer class is mutable sequence of characters. StringBuffer can be used to modify the content of a String with ease. It provides many utility functions to manipulate a string.
- **StringBuilder** − The StringBuilder class is mutable sequence of characters. This provides an API compatible with StringBuffer, but with no guarantee of synchronization.
- **System** − The System class contains several useful class fields and methods. It cannot be instantiated.

Class name	*Brief description*
`ClassLoader`	Subclass of Object and it defines how the classes are loaded
`Compiler`	Subclass of Object, not for normal use; it is used for just-in-time compilation
`Double`	Wrapper class to provide reference to *type* double
`Enum`	Defines methods that support enumeration
`Float`	Wrapper class to provide reference to *type* float
`Integer`	Wrapper class to provide reference to *type* int
`Long`	Wrapper class to provide reference to *type* long
`Math`	Contains several mathematical methods
`Number`	Abstract super class for the Wrapper classes such as Byte, Short, Integer, Float, and Double
`Object`	Super class to all other classes in Java
`Package`	For managing packages
`Process`	For managing processes (programs)
`ProcessBuilder`	Contains methods that provide greater control over processes
`Runtime`	Contains methods to get and use references to objects in runtime environment
`RuntimePermission`	Concerned with Java security mechanism
`SecurityManager`	Supports creation of security manager by subclasses
`Short`	Wrapper class to provide reference to *type* short
`StackTraceElemant`	Defines methods that help trace the class name, file name, etc.

Interfaces defined in java.lang package

1. Appendable

This interface is designed for objects that can have characters or sequences of characters appended to them.

Classes that implement this interface include StringBuilder and BufferedWriter.

2. AutoCloseable

This interface provides a mechanism for objects that need to release resources when they are no longer in use. It has a single method: close().

Commonly used with the try-with-resources statement. Many classes like FileInputStream and Scanner implement this interface.

3. CharSequence

Represents a readable sequence of characters. It is a base interface for several classes like String, StringBuilder, and StringBuffer.

4. Cloneable

This interface indicates that a class can be cloned, meaning it supports the creation of an exact copy of its instances using the clone() method.

Classes implementing this interface can override the clone() method from Object to allow for object duplication.

5. Comparable<T>

This interface is used to impose a natural ordering on objects of each class that implements it. It contains the method compareTo(T o). .

6. Iterable<T>

This interface represents a collection of objects that can be iterated over, typically using a for-each loop.

It defines the method iterator(). Collections like List and Set implement this interface.

7. Readable

This interface represents a source of characters that can be read into a CharBuffer. It has a single method: read(CharBuffer cb).

8. Runnable

This interface is used for classes whose instances are intended to be executed by a thread. It has a single method: run().

It is commonly implemented when creating threads in Java, either directly or indirectly via the Thread class.

Class Object

- It is the super class for all classes in Java. This class has one constructor and 11 methods.

Table 10.6 Methods defined in Object class

Method	*Description*
`Object clone()`	Creates a new copy of invoking object
`boolean equals(Object obj)`	Returns true if invoking object and argument object are equal
`void finalize()`	Executed when the object is garbage collected and the method needs to be overridden by a class
`final Class getClass()`	Identifies the class of invoking object
`int hashCode`	Returns hash code of the invoking object
`final void notify()`	Notifies the thread waiting on invoking thread
`final void notify all()`	Notifies all the threads waiting on invoking thread
`String toString()`	Returns string equivalent of invoking object
`final void wait()`	Invoking thread waits till notified and throws `InterruptedException`
`final void wait((long milliseconds))`	Makes the invoking thread wait for a specified time in milliseconds and throws `InterruptedException`

Class Math

The Math class is an important class in the java.lang package.

It provides methods to perform mathematical calculations.

Some common operations are:

- Trigonometric functions → sin(), cos(), tan()
- Power and exponential functions → pow(), exp()
- Rounding functions → round(), ceil(), floor()
- Other functions → sqrt(), abs(), max(), min()

- Math is a final class, so we **cannot create a subclass of Math**.
  - All methods in the Math class are **static**.
  - Therefore, we **do not need to create an object** of the Math class.
  - But we use the **class name Math** to call its methods.
  - For example:

```
double x = Math.sqrt(25);
System.out.println(x);
```

Method	Description	Return Type	Example
abs(x)	Returns the absolute value of x.	double, float, int, long	Math.abs(-10) → 10
sqrt(x)	Returns the square root of x.	double	Math.sqrt(25) → 5.0
pow(x, y)	Returns x raised to the power y.	double	Math.pow(2, 3) → 8.0
ceil(x)	Returns the smallest integer value greater than or equal to x.	double	Math.ceil(4.2) → 5.0
floor(x)	Returns the largest integer value less than or equal to x.	double	Math.floor(4.8) → 4.0
round(x)	Rounds the value to the nearest integer.	int, long	Math.round(4.6) → 5
max(x, y)	Returns the greater of x and y.	double, float, int, long	Math.max(10, 20) → 20
min(x, y)	Returns the smaller of x and y.	double, float, int, long	Math.min(10, 20) → 10
random()	Returns a random value from 0.0 (inclusive) to 1.0 (exclusive).	double	Math.random() → 0.735
sin(x)	Returns the sine of x, where x is in radians.	double	Math.sin(0) → 0.0
cos(x)	Returns the cosine of x, where x is in radians.	double	Math.cos(0) → 1.0
tan(x)	Returns the tangent of x, where x is in radians.	double	Math.tan(0) → 0.0

Wrapper Classes

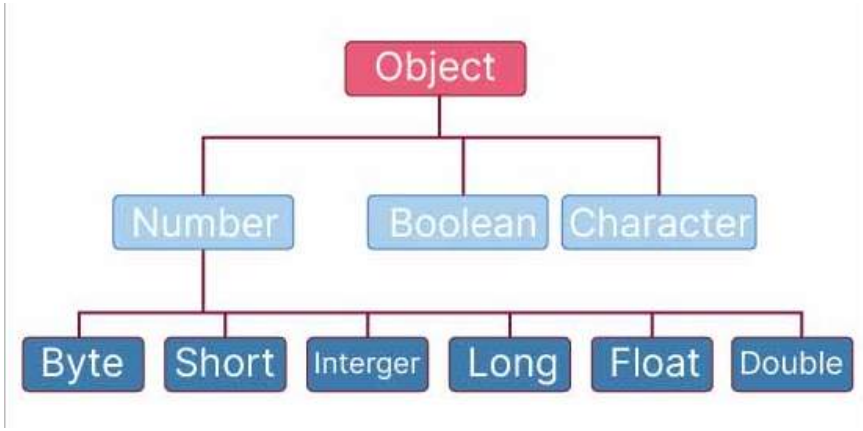
- In Java, everything can be viewed as an object.
- However, primitive data types such as int, char, double, and boolean are not objects.
- Java provides Wrapper Classes to convert primitive data types into objects.
- A wrapper class wraps or encloses a primitive value inside an object.
- Wrapper classes are available in the java.lang package.
- Wrapper class objects are used to pass arguments to collection framework methods.

Primitive Type	Wrapper Class	Example
byte	Byte	Byte b = 10;
short	Short	Short s = 20;
int	Integer	Integer i = 100;
long	Long	Long l = 1000L;
float	Float	Float f = 10.5f;
double	Double	Double d = 25.5;
char	Character	Character c = 'A';
boolean	Boolean	Boolean b = true;

Wrapper Classes

- The eight primitive data types, namely boolean, byte, short, int, long, float, double, and char, are not objects;
- They cannot be passed by references and they are passed by value only.
- In order to provide object representation, eight Wrapper classes are defined in java.lang package.

Pimitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean



Constructors of Wrapper Classes

class Byte

Byte (byte *Numb*)

Byte (String *Strnumb*) throws NumberFormatException

class Short

Short (short *numb*)

Short (String *Strnumb*) throws NumberFormatException

class Integer

Integer (int *numb*)

Integer (String Strnumb) throws NumberFormatException.

class Float

Float (float numb or double number)

Float (String Strnumb) throws NumberFormatException

The NumberFormatException occurs when the string is not in appropriate format.

Constructors of Wrapper Classes

class Double

Double(double d)

Double (String *Strnumb*) throws NumberFormatException

class Character

Character (char ch)

class Boolean

Boolean (boolean value)

Boolean (String boolstring)

Java has deprecated these constructors in favor of using valueOf() or autoboxing.
First see Ex1.java , then see UseofValueOf.java example program

Methods of Wrapper Classes

- The valueOf() method is a static method available in wrapper classes such as Integer, Double, Float, and Boolean.
- It converts a primitive value or a String into an object of the corresponding wrapper class.
- Syntax:
  - static T valueOf(type number)
  - static T valueOf(String number)

T represents the wrapper class return type.
type number represents a primitive value.
String number represents a string containing a value.

UseofValueOf.java

Methods of Wrapper Classes

- **Integer Class**
  - public static Integer valueOf(int i)
  - public static Integer valueOf(String s)

- **Double Class**
  - public static Double valueOf(double d)
  - public static Double valueOf(String s)

Boolean Class

public static Boolean valueOf(boolean b)
public static Boolean valueOf(String s)

Methods of Wrapper Classes

Method	Description
byte byteValue()	is used to convert an object of a wrapper class (such as Integer, Float, Double, etc.) to a byte primitive type.
int intValue()	Returns the value as an int primitive type.
long longValue()	Returns the value as a long primitive type.
short shortValue()	Returns the value as a short primitive type.
double doubleValue()	Returns the value as a double primitive type.
float floatValue()	Returns the value as a float primitive type.

These methods are defined in the Number class (which is the superclass of classes like Integer, Long, Float, etc.), so they are available in various wrapper classes.

String toString()	Returns the string equivalent of invoking object

Ex2.java

Methods for Conversion of Digital Strings into numbers

- Parse methods convert a String containing a valid number into its corresponding primitive data type.
- These static methods belong to wrapper classes such as Integer, Double, Float, Byte, Short, and Long.

Method	Wrapper Class	Converts String to Primitive
`parseInt()`	`Integer`	`int`
`parseFloat()`	`Float`	`float`
`parseDouble()`	`Double`	`double`
`parseByte()`	`Byte`	`byte`
`parseShort()`	`Short`	`short`
`parseLong()`	`Long`	`long`

Ex2.java

Table 10.12 Methods for conversion of digital strings into numbers

Method	*Description*
`static float parseFloat(String str)`	Returns float equivalent of the number in digital string str. The radix value is 10.
`static double parseDouble (String str)`	Returns double equivalent of the number in digital string str. The radix value is 10.
`Static byte parseByte(String str)`	Returns byte equivalent of the number in digital string str. The radix value is 10.
`static short parseShort(String str)`	Returns short equivalent of the number in digital string str. The radix value is 10.
`static int parseInteger(String str)`	Returns integer equivalent of the number in digital string str. The radix value is 10.
`static long parseLong (String str)`	Returns long equivalent of the number in digital string str. The radix value is 10.

Auto-boxing and Un-boxing

Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper class object.

Examples:

int → Integer
double → Double
float → Float
boolean → Boolean
long → Long

Ex: int a = 100;
Integer obj = a;

Autoboxing and unboxing were introduced in Java 5 (JDK 1.5).

Unboxing

- Unboxing is the automatic conversion of a wrapper class object into its corresponding primitive data type.
- Ex:

```
Integer obj = 200;
int b = obj;
```

Remember:
Autoboxing: Primitive to object.
Unboxing: Object to primitive.

Methods isInfinite() and isNaN()

- The methods isInfinite() and isNaN() are used in Java to check if a floating-point value is either infinite or "Not-a-Number" (NaN).
- These methods are available in the Double and Float wrapper classes.
- isInfinite() is occurer due to large number/small number
  - Ex: 252256897771/2, number/0
- isNaN() checks whether the invoking object is number or not

As per textbook

Method	*Description*
`boolean isNaN()`	Returns true if the invoking object is not a number, otherwise false
`byte byteValue()`	Returns value of invoking object as byte
`int compareTo(type object)`	Compares the value of invoking object with that of the argument object and returns 0 or a positive value or a negative value, respectively; if the two are equal, invoking object has greater value or invoking object has lower value than the argument object
`Double doubleValue()`	Returns value of invoking object as double
`Float floatValue()`	Returns value of invoking object as float
`int hashCode()`	Returns the hashCode for the invoking object
`int intValue()`	Returns value of invoking object as int
`long longValue()`	Returns value of invoking object as long
`short shortValue()`	Returns value of invoking object as short
`static String toHexString(type numbr)`	Returns string containing value of number in hexadecimal format. This method is not there in Byte and Short classes
`static T valueOf(type numbr)`	Returns the type of object containing value passed on by number
`static T valueOf(String str)`	Returns the object of type T containing value specified by digit String str
`String toString(type numbr)`	Returns the string equivalent of invoking object

Exception Handling

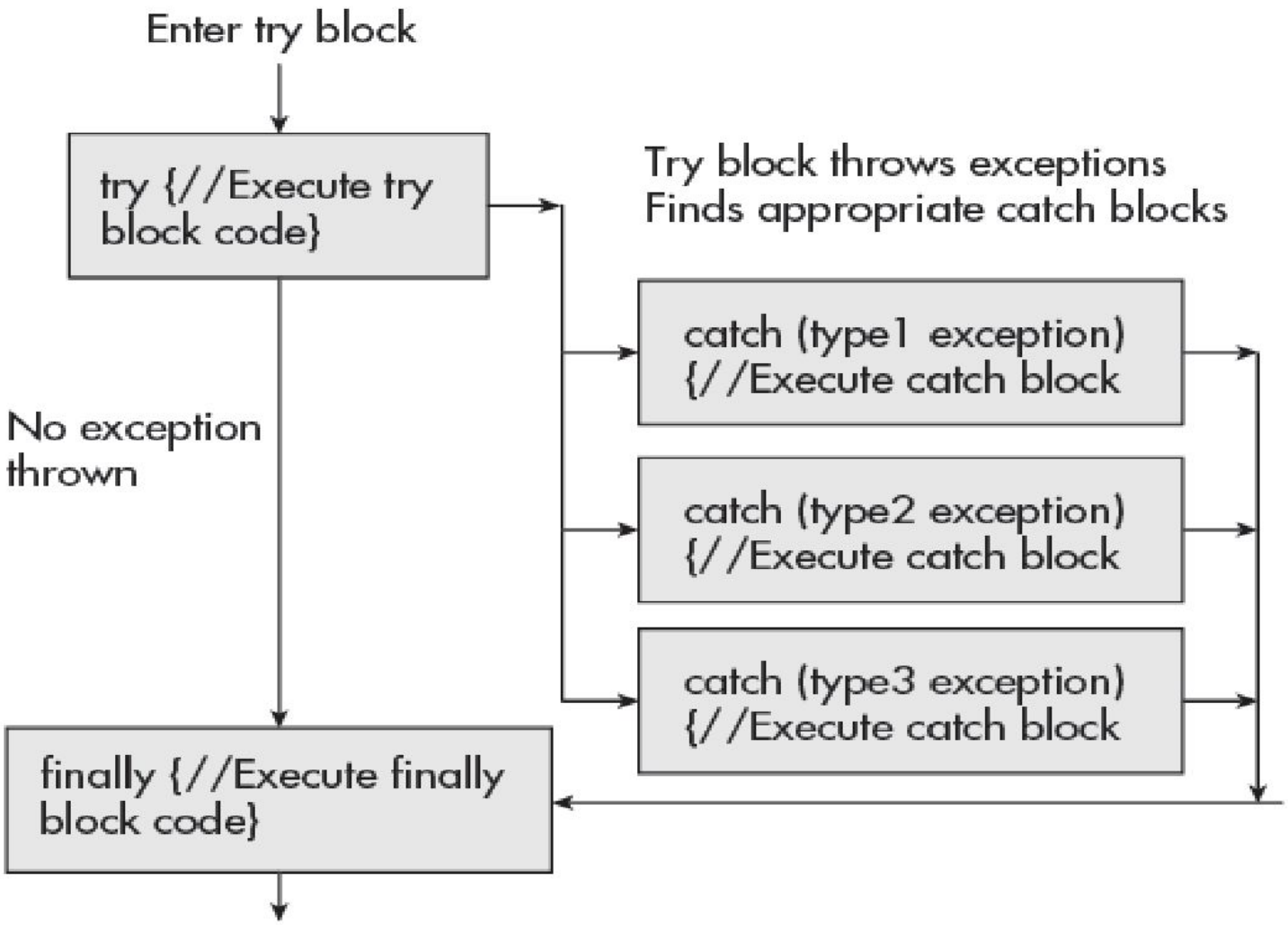
- types of errors
  - Compile, run-time, logical errors(incorrect o/p)
    - Logical Errors
    - Multiplying when you should be dividing
    - Adding when you should be subtracting
    - Opening and using data from the wrong file
    - Displaying the wrong output

- Exceptions – occurred during running and classified into checked or unchecked.
- An exception is undesirable that may occur during the execution of the program and may lead to termination of the program if it is not handled properly.
    - try
    - catch
    - throw - explicitly raise
    - throws – not in a position to handle
    - finally – always execute irrespective of exception occurred or not

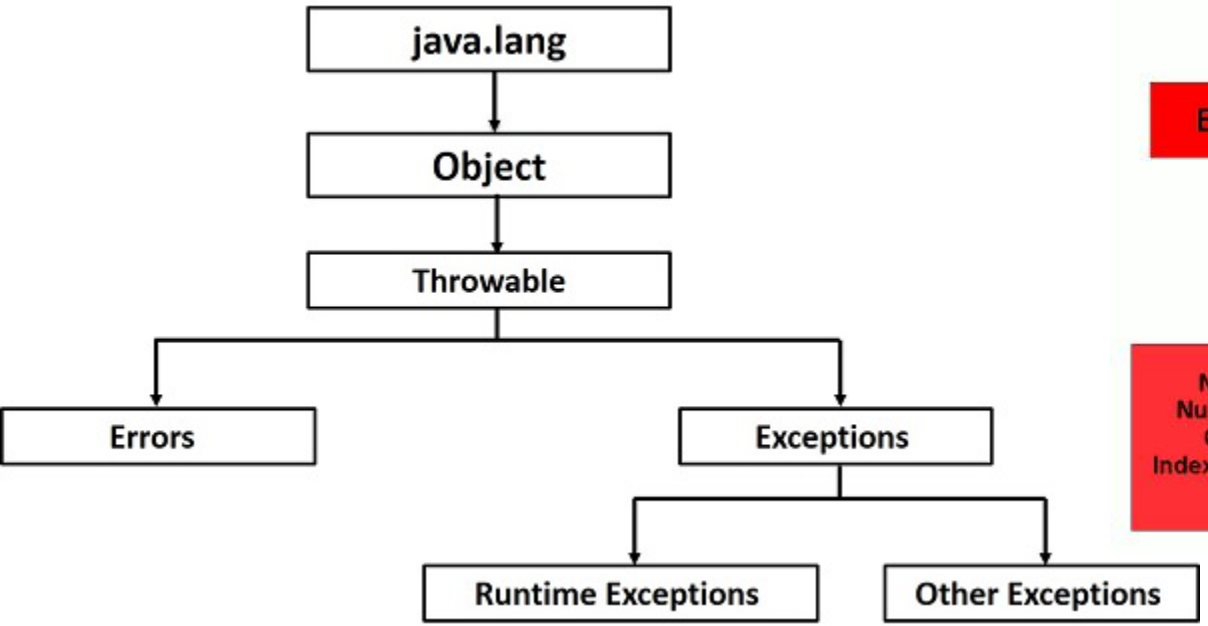
General form-of exception handling block

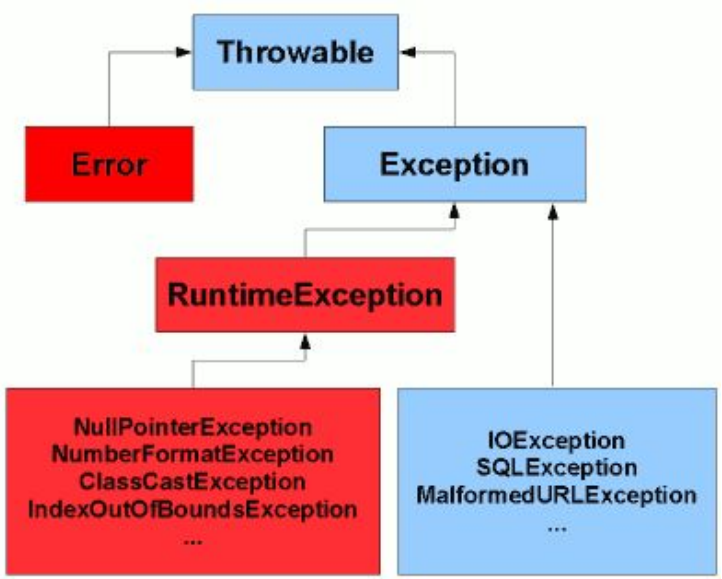
```
try {
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb) {
    // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
    // exception handler for ExceptionType2
}
// ...
finally {
    // block of code to be executed after try block ends
}
```

- Here, *ExceptionType is the type of exception that has occurred.*

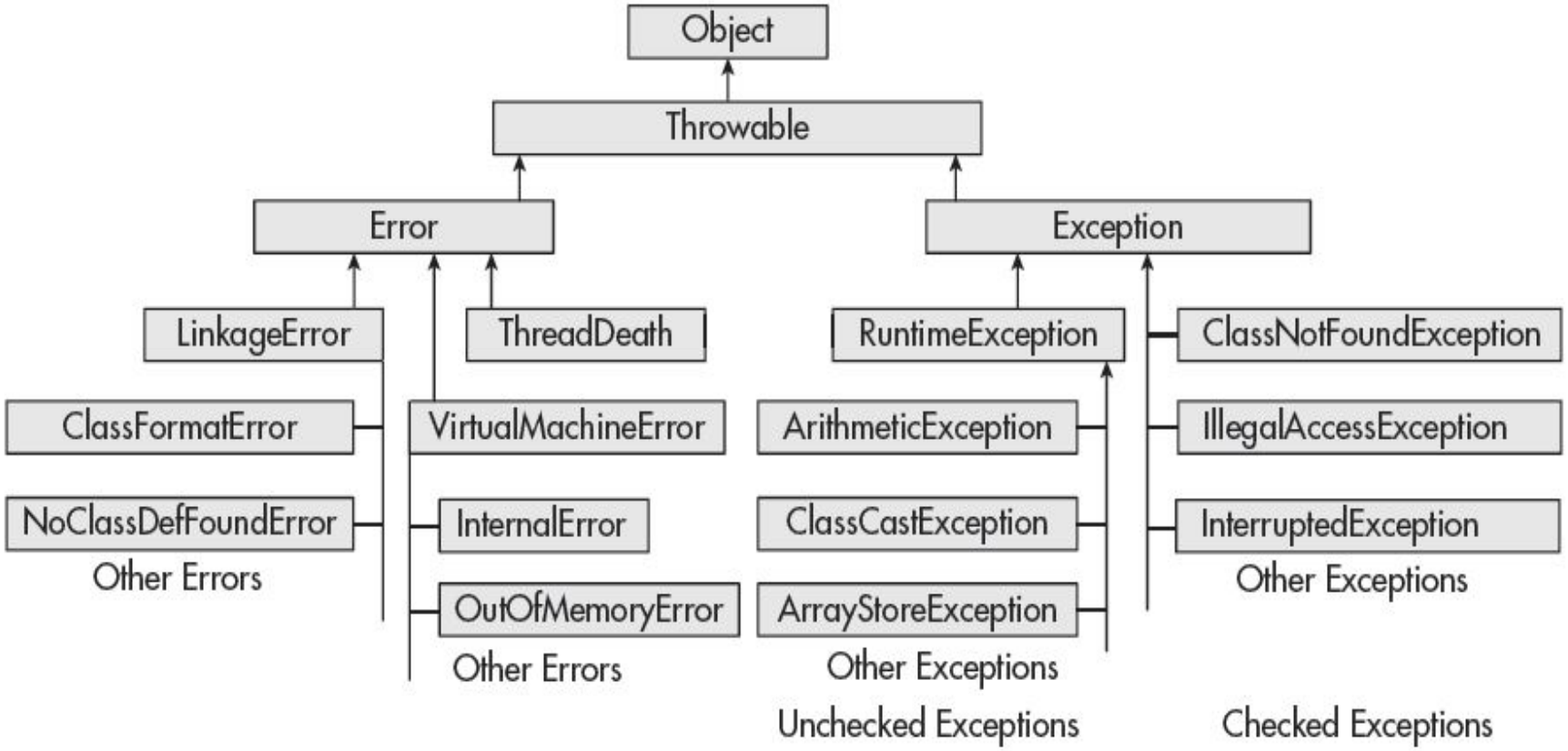

Enter try block
try {//Execute try block code}
Try block throws exceptions
Finds appropriate catch blocks
catch (type1 exception) {//Execute catch block
catch (type2 exception) {//Execute catch block
catch (type3 exception) {//Execute catch block
No exception thrown
finally {//Execute finally block code}

Exception heirarchy





Error - Out of memory error or a System crash error

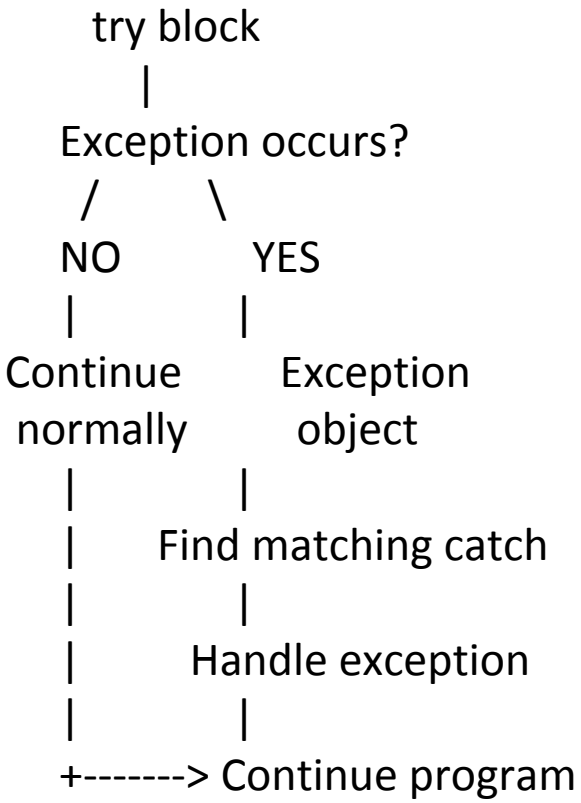

Object
Throwable
Error
Exception
LinkageError
ThreadDeath
RuntimeException
ClassNotFoundException
ClassFormatError
VirtualMachineError
ArithmeticException
IllegalAccessException
NoClassDefFoundError
InternalError
ClassCastException
InterruptedException
Other Errors
OutOfMemoryError
ArrayStoreException
Other Exceptions
Other Errors
Other Exceptions
Unchecked Exceptions
Checked Exceptions

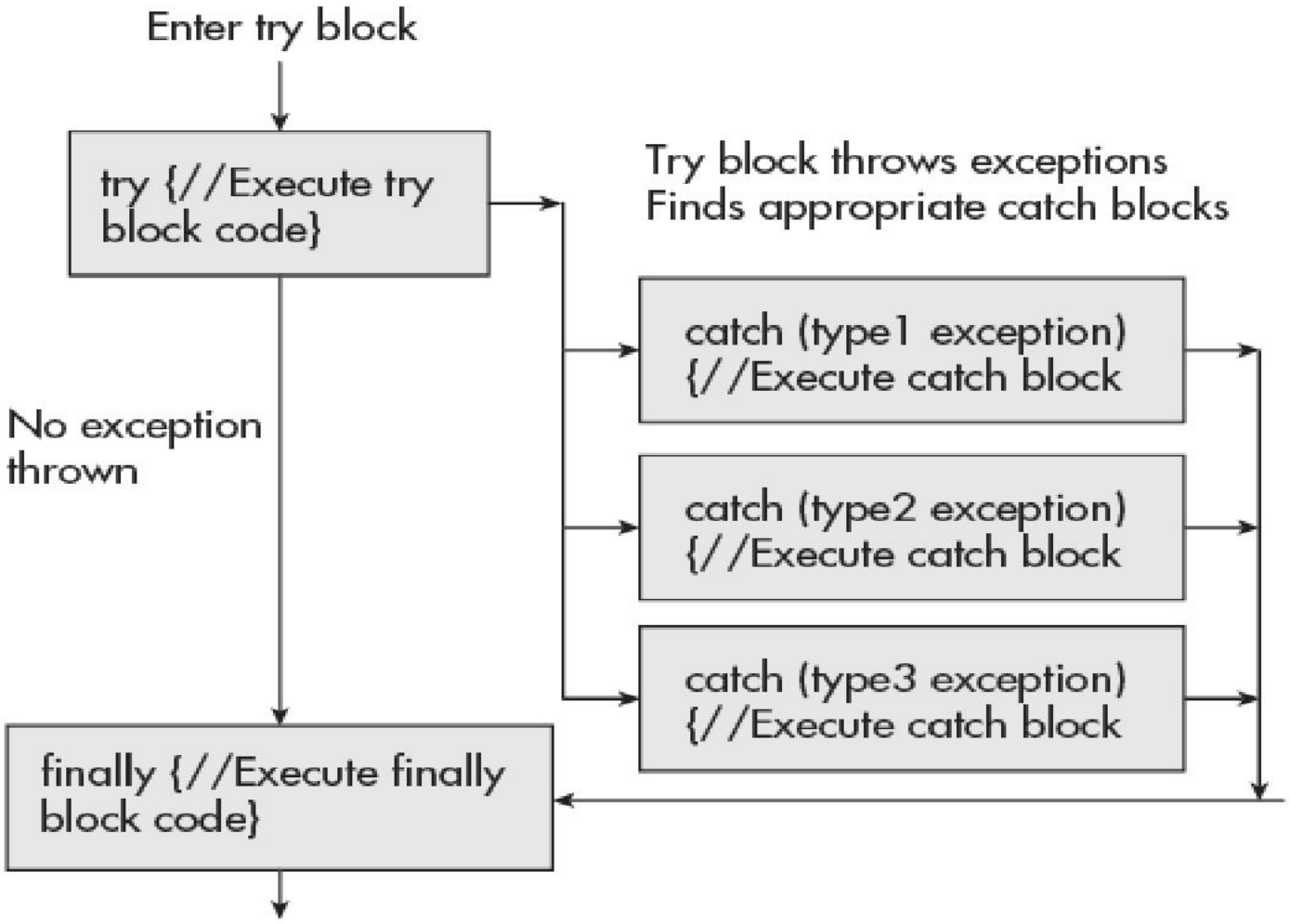
FAQs

- What is Throwable?
  - Throwable is a super class that represents all errors and exceptions
- Which is the super class for all exceptions?
- What is the difference between exception & error?
  - An exception is an error which can be handled. It means when an exception happens, the programmer can do something to avoid harm.
  - But an error is an error which can not be handled (Programmer can not do anything –catastrophically errors)
- What are checked exceptions?
  - That are checked at compilation time by the Java compiler - JVM
- What are unchecked exceptions?
  - That are checked by JRE

Working of try-catch

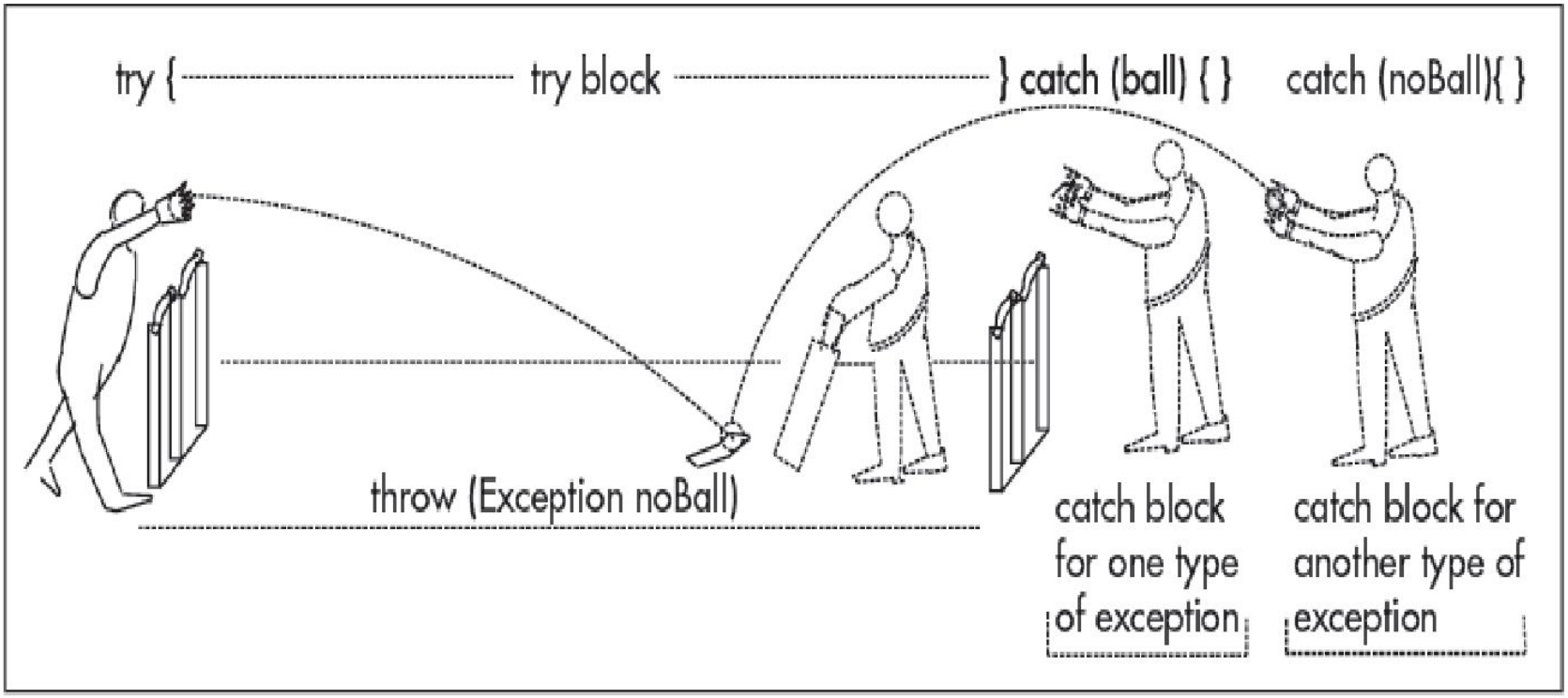
- **try block** contains the code that may cause an exception.
- **catch block** is used to handle the exception if it occurs.
- **If no exception occurs:**
  - All statements in the try block execute normally.
  - The catch block is skipped.
  - Program execution continues **after the catch block(s)**.
- **If an exception occurs:**
  - Java creates an **exception object**.
  - The exception object contains information about the exception, including its **type**.
- The remaining statements in the try block are **not executed**.
- Java comes out of the try block and searches for a **matching catch block**.
- If a suitable catch block is found, the exception is **handled** there.


try block
Exception occurs?
NO
YES
Continue normally
Exception object
Find matching catch
Handle exception
Continue program


Enter try block
try {//Execute try block code}
Try block throws exceptions
Finds appropriate catch blocks
catch (type1 exception) {//Execute catch block
No exception thrown
catch (type2 exception) {//Execute catch block
catch (type3 exception) {//Execute catch block
finally {//Execute finally block code}

catch block

- A catch block handles an exception when the **exception type matches** the type specified in the catch block.
- If the first catch block does **not match**, Java checks the **next catch block**.
- This checking continues **from top to bottom** until a matching catch block is found.
- When a matching catch block is found, its statements are executed, and the remaining catch blocks are skipped.
- If **no catch block matches**, the exception remains **uncaught**.
- If a finally block is present, it executes before the program terminates due to the uncaught exception.


try {
try block
} catch (ball) { }
catch (noBall){ }
throw (Exception noBall)
catch block for one type of exception
catch block for another type of exception

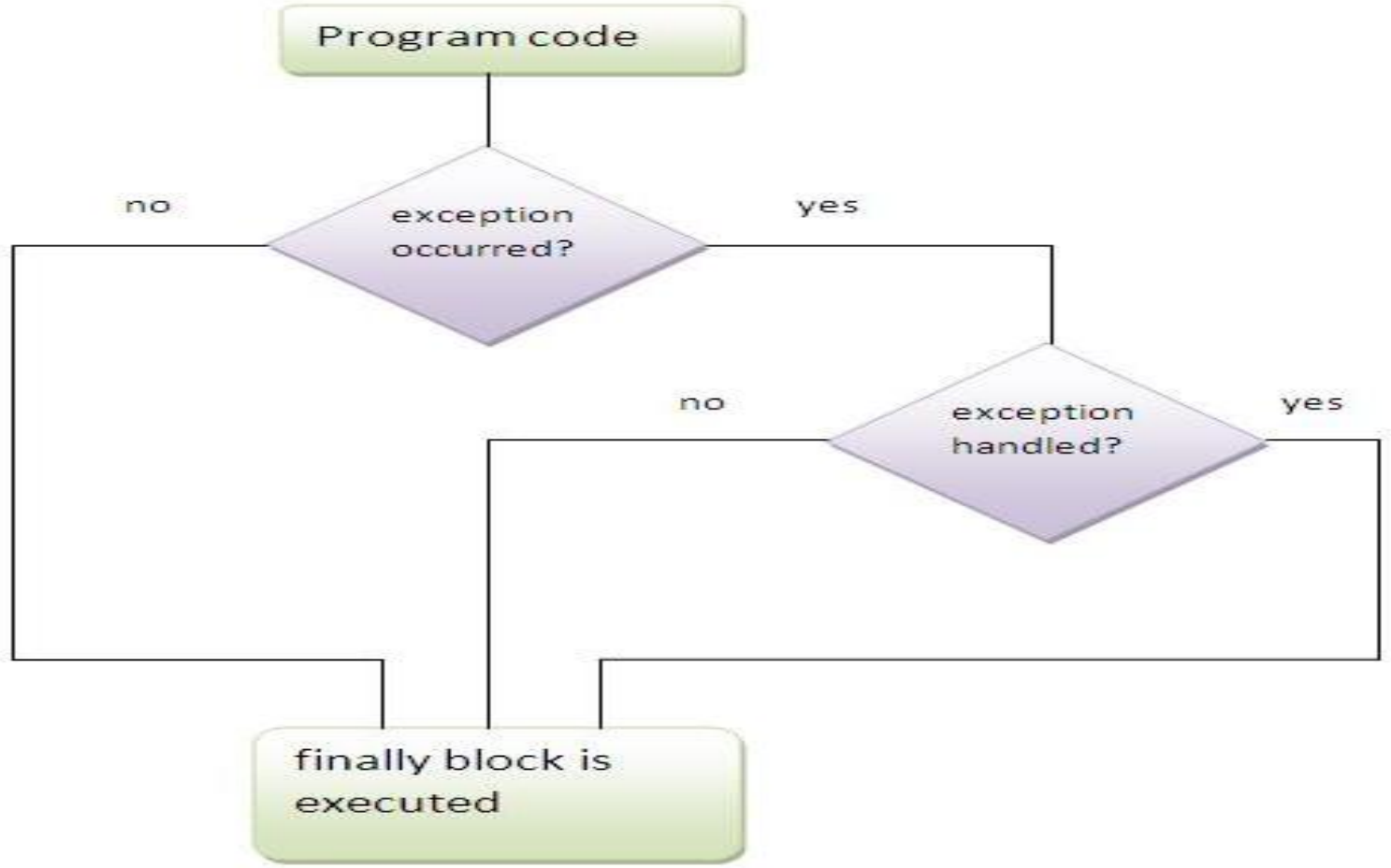
Multiple catch clauses

- In some cases, more than one exception could be raised by a single piece of code.
- To handle this type of situation, you can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- If one catch statement is executed, the others are bypassed, and execution continues after the try / catch block.

- When you use multiple catch statements, it is important to remember that exception subclasses must come before any of their super classes.
- This is because a catch statement that uses a super class will catch exceptions of that type plus any of its subclasses. Subclass would never be reached if it came after its super class.
- Since Java 7, a modification has been done in the language so that one catch block can catch more than one type of exception.
  - The exception classes are connected by Boolean | (or) operator |.
  - Ex: catch (ArithmeticException | NullPointerException e) { }

Java finally block

- **Java finally block** is a block that is used *to execute important code* such as closing connections (databases, network, disks, commit in databases) etc.
- Java finally block is always executed whether exception is handled or not.


Program code
exception occurred?
no
yes
exception handled?
no
yes
finally block is executed

Use of throw – to generate and for rethrowing exceptions

- So far, you have only been catching exceptions that are thrown by the Java run-time system. However, it is possible for your program to throw an exception explicitly, using **throw** statement.
- The general form of throw is shown here:

  throw ThrowableInstance;

- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.

- There are two ways to obtain a Throwable instance:
  - creating one with the new operator
    throw new exception_class("error message");
  - using the parameter of catch clause -
    throw exception_type_variable; (For rethrowing)

- The flow of execution stops immediately after the throw statement; any subsequent statements are not executed.
- The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception.
- If it does find a match, control is transferred to that statement.
- If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

Use of throws clause

- throws is a **keyword in Java** used at the **method declaration/signature**.
- It indicates that a method **may throw one or more exceptions**.
- It is mainly used to declare **checked exceptions**. It can also list unchecked exceptions, but declaring unchecked exceptions is not mandatory.
- The **caller of the method** must handle a declared checked exception using **try-catch** or further declare it using throws.

```
type method-name(parameter-list) throws exception-list
{
 // body of method
}
```

Custom or Userdefined Exceptions

- We can also create our own exception by creating a sub class simply by extending java **Exception** class.

  class UserException extends Exception

- Define a constructor in **UserException** (optional)
  - **UserException**(); -> Creates an exception **without a description/message**.
  - **UserException** (parameter); -> Creates an exception **with a custom description/message**

- To print custom messages:
  - Override **toString()** method
  - **toString()** method to display customized message in catch clause.

Class Throwable

- The class is declared as

  public class Throwable extends Object

  implements serializable

- Throwable is the root of Java's error and exception hierarchy.
- Only **Throwable objects** (or subclasses) can be thrown with **throw**.
- Only **Throwable types** can appear as the parameter of a **catch** clause.

Constructors of class Throwable

- **Throwable()**: the detail message is null and no cause is set.
- **Throwable(String message)**: stores your message, retrievable with getMessage().
- **Throwable(String message, Throwable cause)**: used for **exception chaining**, where you wrap a low-level exception inside a higher-level one and keep both messages.
- **Throwable(Throwable cause)**: the message is automatically set to cause.toString() (or null if the cause is null).

```
Throwable t1 = new Throwable(); // message = null
Throwable t2 = new Throwable("Something failed"); // message set
Throwable t3 = new Throwable("Wrapper", cause); // message + cause
Throwable t4 = new Throwable(cause); // cause only
```

Methods

String getMessage()	Returns the detail message
String toString()	Returns the class name, plus ": " and the message if one exists.
Throwable getCause()	Returns the cause, or null if none or unknown.
void printStackTrace()	Prints the exception and its stack trace to standard error (System.err).
StackTraceElement[] getStackTrace()	Returns the trace as an array; each element holds class, method, file and line number.

Methods of class Throwable

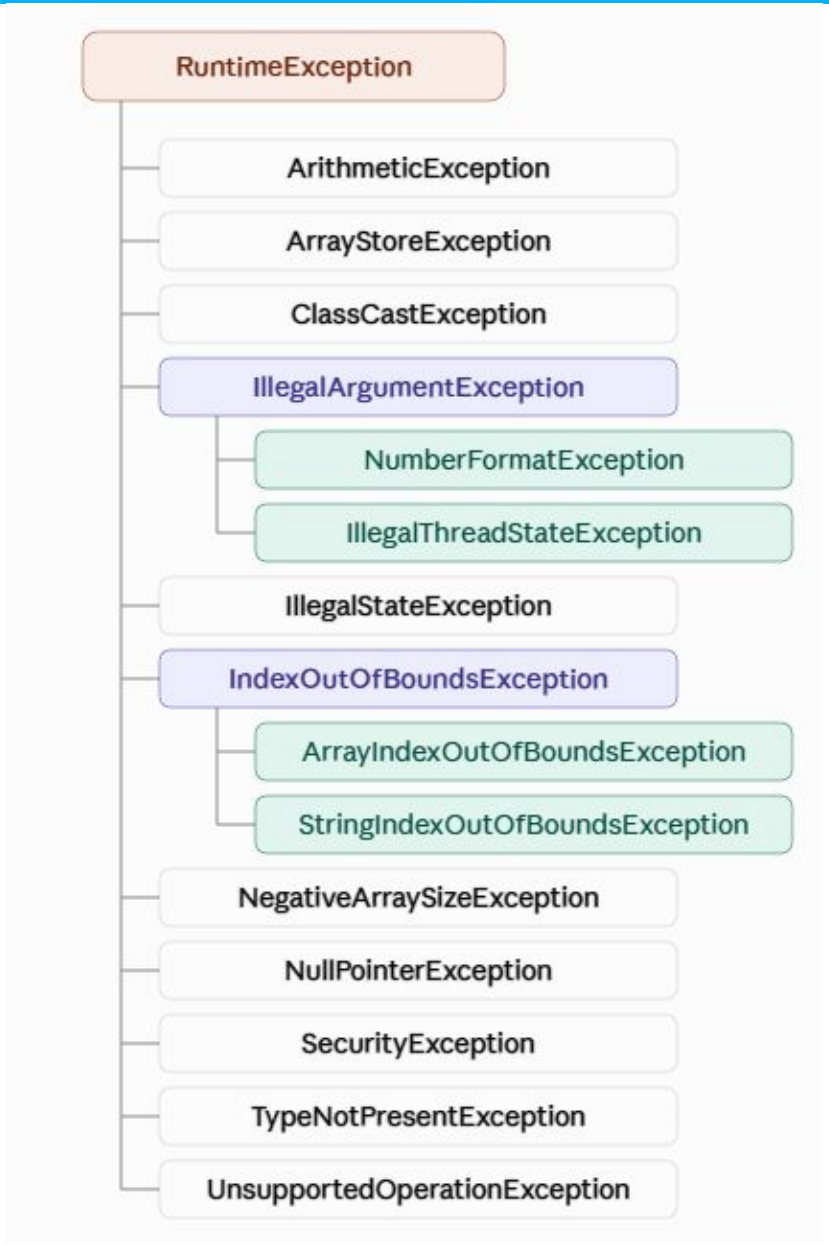
Table 11.3 Commonly used methods defined in class `Throwable`

Method name	*Description*
`void addSuppressed( Throwable exception)`	The specified exception is appended to exceptions that were suppressed.
`Throwable fillInStackTrace()`	The method fills in execution stack trace.
`Throwable getCause()`	It returns the cause of the throwable, otherwise false if not available.
`String getMessage ()`	The method returns a detailed string message that gives information about the current exception.
`String getLocalizedMessage()`	It returns a string version of a localized description of this throwable.
`StackTraceElement[] getStackTrace()`	It gives access to StackTraceElement.
`Throwable [] getSuppressed()`	It returns an array of all exceptions suppressed by try-with-resources.
`Throwable initCause( Throwable cause)`	It initializes with specified value of the cause of this throwable.
`void printStackTrace()`	It traces and displays the hierarchy of method calls that resulted in exception.
`void printStackTrace( printStream strm)`	It prints this throwable and its back trace to standard error stream.
`void printStackTrace( printWriter strm)`	The method prints this throwable and the trace to the specified print writer.
`void setStackTrace( StackTraceElement[] stackTrace)`	The method sets the stack trace elements.
`String toString()`	The class Throwable overrides the method `toString()` of object class for displaying a short message about this throwable.

Unchecked Exceptions

- The unchecked exception classes are subclasses of class RuntimeException, which is further a subclass of class Exception.
- The runtime exceptions are not checked by the compiler.
- These exceptions form a subgroup of classes that are subclasses of class Exception.

Unchecked subclasses of RuntimeException



Exception	Typical trigger
ArithmeticException	int x = 10 / 0;
ArrayIndexOutOfBoundsException	int[] a = new int[3]; a[5] = 1;
ArrayStoreException	Object[] o = new String[2]; o[0] = 10;
ClassCastException	Object o = "hi"; Integer i = (Integer) o;
IllegalArgumentException	Passing an invalid value, e.g. Thread.sleep(-1)

IllegalStateException	Calling a method at the wrong time, e.g. iterator.remove() before next()
	Iterator<String> it = list.iterator();
	it.next(); // Move to the first element
	it.remove();

IllegalThreadStateException	Calling start() twice on the same thread

Exception	Typical trigger

IndexOutOfBoundsException	List<String> list = new ArrayList<>();
	list.add("A"); list.add("B"); list.add("C");
	
	list.get(10);

NegativeArraySizeException	int[] a = new int[-5];
NumberFormatException	Integer.parseInt("abc");
NullPointerException	String s = null; s.length();
SecurityException	Code violating a security manager's rules

Checked Exceptions

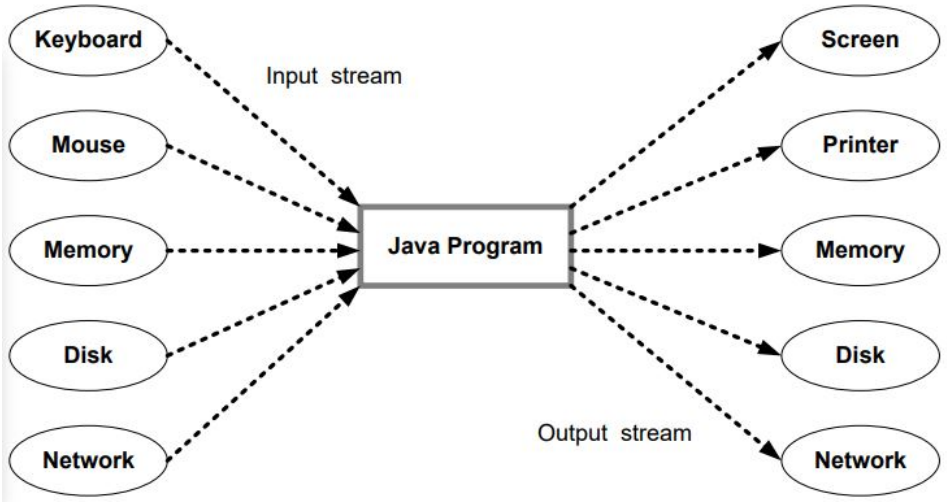
- The compiler forces you to either catch them with try-catch or declare them with throws.
- They represent problems that can happen even in correct code, usually because of the outside environment (missing files, missing classes, interrupted threads).

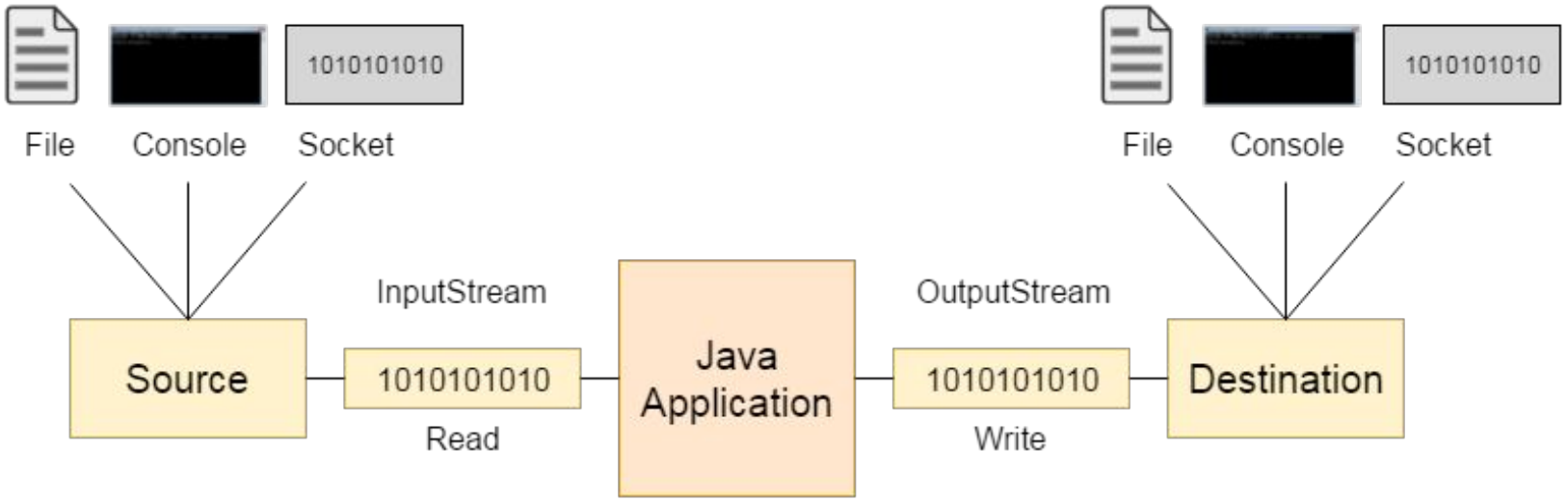
Exception	Typical trigger
ClassNotFoundException	Class.forName("com.missing.Foo"); //is used to **load a class into the JVM at runtime**.
FileNotFoundException	new FileReader("missing.txt");
IllegalAccessException	Using reflection to access a private member or class without permission
InstantiationException	Class.forName("MyAbstractClass").newInstance(); via reflection
InterruptedException	Thread.sleep(1000); when another thread interrupts it
NoSuchFieldException	MyClass.class.getField("nofield");
NoSuchMethodException	MyClass.class.getMethod("nomethod");

- **Java I/O and File:** Java I/O API, standard I/O streams, types, Byte streams, Character streams, Scanner class, Files in Java: File, FileInputStream and FileOutputStream Classes

Stream in Java

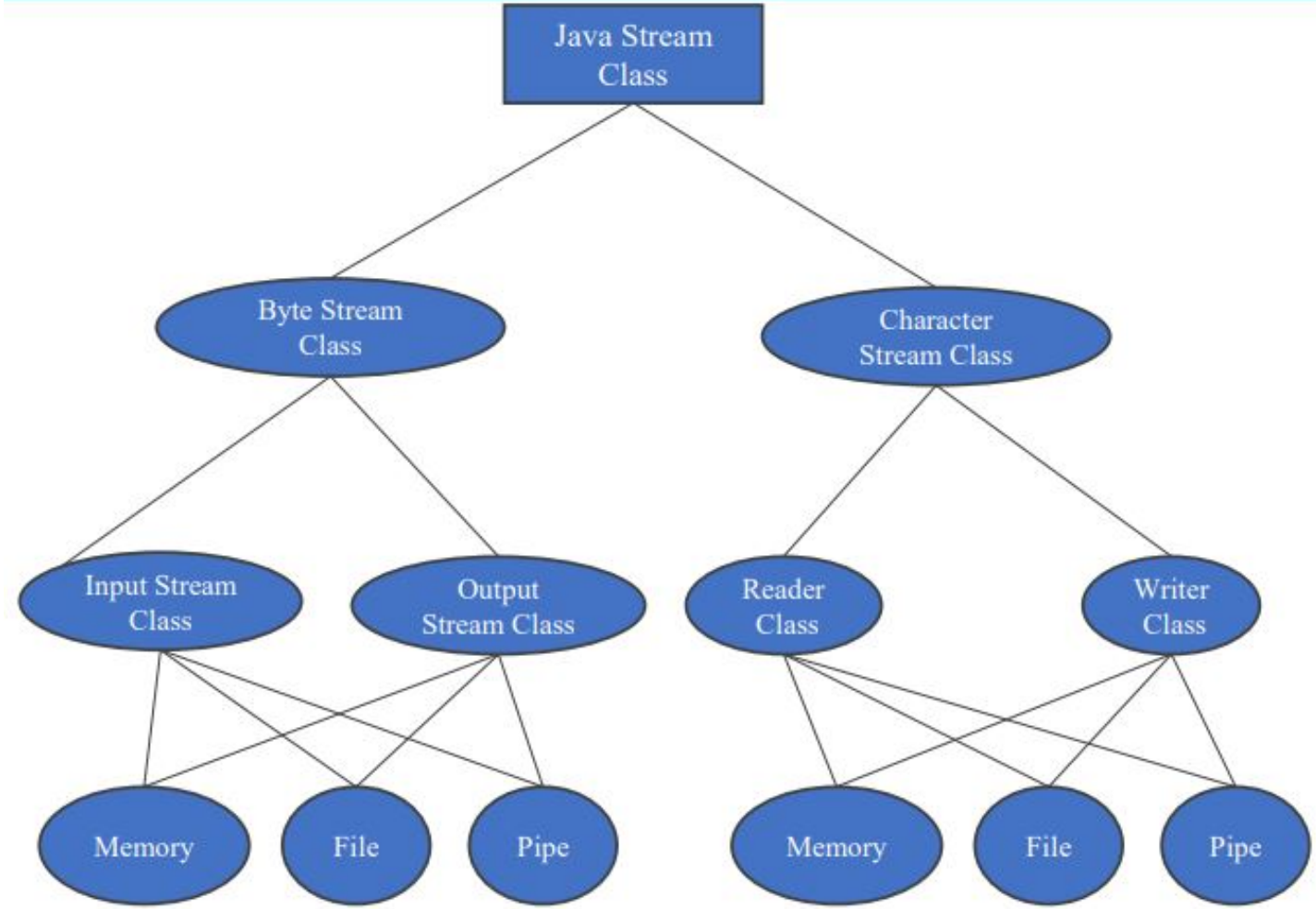
- Java treats flow of data as stream.
- Java streams are classified into two basic types, namely, input stream and output stream.
- The java.io package contains a large number of stream classes to support the streams.




1010101010
File
Console
Socket
1010101010
File
Console
Socket
InputStream
OutputStream
Source
1010101010
Read
Java Application
1010101010
Write
Destination

I-O stream classes in Java

- Java provides java.io package which contains a large number of stream classes to process all types of data
- Byte stream classes
  - Support for handling I/O operations on bytes
- Character stream classes
  - Supports for handling I/O operations on characters


Java Stream Class
Byte Stream Class
Character Stream Class
Input Stream Class
Output Stream Class
Reader Class
Writer Class
Memory
File
Pipe
Memory
File
Pipe

- Java provides a rich set of classes for performing I/O operations. Some of the key classes include:
  - **InputStream and OutputStream:** These abstract classes form the foundation for byte-oriented I/O operations. They provide methods for reading and writing bytes from/to various sources and destinations.
  - **Reader and Writer:** These abstract classes are used for character-based I/O operations. They provide methods for reading and writing characters from/to character-based streams.
  - **FileInputStream and FileOutputStream:** These classes allow reading from and writing to files in a byte-oriented manner.
  - **FileReader and FileWriter:** These classes enable reading from and writing to files using character-oriented operations.
  - **BufferedInputStream and BufferedOutputStream:** These classes provide buffering capabilities, which can significantly improve I/O performance by reducing the number of system calls.
  - **BufferedReader and BufferedWriter:** These classes offer buffered reading and writing of character data, enhancing I/O efficiency when working with character-based streams.

- In Java, 3 streams are created for us automatically.
- All these streams are attached with the console (The console is typically the terminal or command line where you can input data and see output).

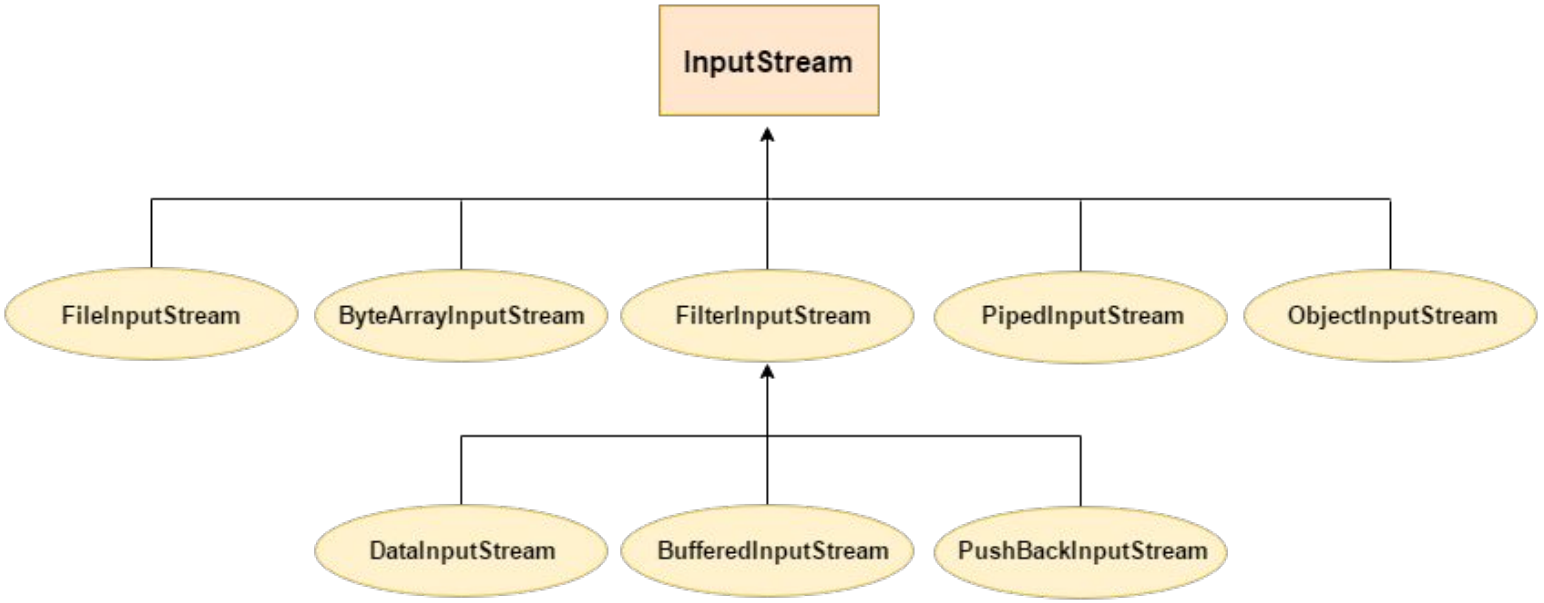
  1) System.out: standard output stream

  2) System.in: standard input stream

  3) System.err: standard error stream

Example programs to read data StandardInputDemo.java, ConsoleDemo.java

Java Input Stream Classes



- FilterInputStream is an abstract class
- ByteArrayInputStream – reads the data in terms of bytes (images, audio files…)

Important Points

FileInputStream	Reading from a file
ByteArrayInputStream	Reading from memory
FilterInputStream	Additional functionality
PipedInputStream	Receiving data between threads
ObjectInputStream	Reading objects
DataInputStream	Reading primitive data
BufferedInputStream	Faster reading using a buffer
PushbackInputStream	Pushing back and rereading bytes

InputStream

Base class for reading bytes from any source.

FileInputStream



Reads bytes from a file.

ByteArray InputStream



Reads bytes from a byte array in memory.

FilterInputStream



Adds extra features to another input stream.

PipedInputStream



Receives data from another thread through a pipe.

ObjectInputStream



Reads serialized Java objects.

DataInputStream

int float double

Reads primitive data types like int, float and double.

BufferedInputStream



Improves reading speed using a buffer.

PushbackInputStream



Allows bytes to be pushed back and read again.

Key Takeaways

- **FileInputStream** → File reading
- **ByteArrayInputStream** → Memory reading
- **ObjectInputStream** → Object reading
- **DataInputStream** → Primitive data reading
- **BufferedInputStream** → Faster reading
- **PushbackInputStream** → Read again
- **PipedInputStream** → Inter-thread communication

Java input stream classes

InputStream classes is used to read 8-bit bytes and supports a number of input-related methods

- Reading bytes
- Closing streams
- Marking positions in streams
- Skipping ahead in streams
- Finding the number of bytes in stream
- and many more…

Useful Methods of InputStream Class

Method	Description
`read( )`	Read a byte from the input stream
`read(byte b[ ])`	Read an array of bytes into b
read(byte b[ ], int n, int m)	Reads m bytes into b starting from n^{th} byte
`available( )`	Gives number of bytes available in the input
`skip(n)`	Skips over n bytes from the input stream
`reset( )`	Goes back to the beginning of the stream
`close( )`	Close the input steam

Example:

DataInputStream

readShort( )	**readDouble( )**
readInt( )	readLine( )
readLong( )	readChar( )
readFloat( )	readBoolean( )
readUTF( )	

Useful Methods of InputStream Class

- **Reading bytes**
  - int read()
  - int read (byte b[]) – read an array of bytes into an array
  - int read (byte b[], int off, int len)
- **Closing streams**
  - void close()
- **Finding the number of bytes in a stream**
  - int available()
- **Skipping ahead in a stream**
  - long skip (long n)
- **Marking positions in a stream**
  - void mark (int limit)
  - void reset()
  - boolean markSupported()

Java InputStream Class – Important Methods

InputStream is used to read data from a file or other source in the form of bytes.

1. Reading Bytes

int read()	• Reads one byte at a time. • Returns byte value as int, -1 at end of stream.	`int data = inputStream.read();`
int read(byte b[])	• Reads multiple bytes. • Stores in given array. • Returns count.	`byte b[] = new byte[5];` `int count = inputStream.read(b);`
int read(byte b[], int off, int len)	• Reads specified number of bytes. • Stores from index off, maximum len.	`byte b[] = new byte[10];` `inputStream.read(b, 2, 5);`

2. Closing Streams

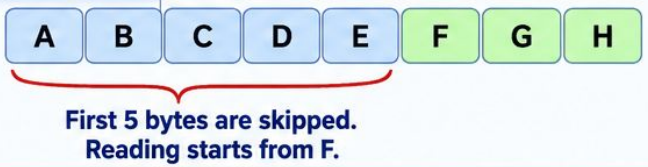
void close()	• Closes the input stream. • Releases resources.	`inputStream.close();`

3. Finding the Number of Bytes

int available()	• Returns estimated number of bytes that can be read without blocking.	`int count = inputStream.available();`

4. Skipping Ahead in a Stream

long skip(long n)	• Skips n bytes. • Returns actual bytes skipped.	`inputStream.skip(5);`



5. Marking Positions in a Stream

void mark(int limit)	• Marks current position. • Limit = how many bytes can be read before mark may become invalid.	`inputStream.mark(100);`
void reset()	• Returns to previously marked position.	`inputStream.reset();`
boolean markSupported()	• Checks if mark/reset is supported. • Returns true or false.	`System.out.println(inputStream.markSupported());`

Quick Revision Table

Method	Purpose	Method	Purpose
read()	Reads one byte	available()	Estimates bytes readable without blocking
read(byte b[])	Reads multiple bytes	skip(n)	Skips bytes
read(byte b[], off, len)	Reads bytes into a specified array position	mark(limit)	Remembers current position
close()	Closes the stream	reset()	Returns to marked position
		markSupported()	Checks mark/reset support

Key Takeaways

- ✅ **mark()** = Remember the position.
- ✅ **reset()** = Return to the remembered position.
- ✅ **skip()** = Move forward by skipping bytes.
- ✅ Not all InputStream implementations support mark() and reset(). Always check using **markSupported()**.

DataInputStream Class Methods

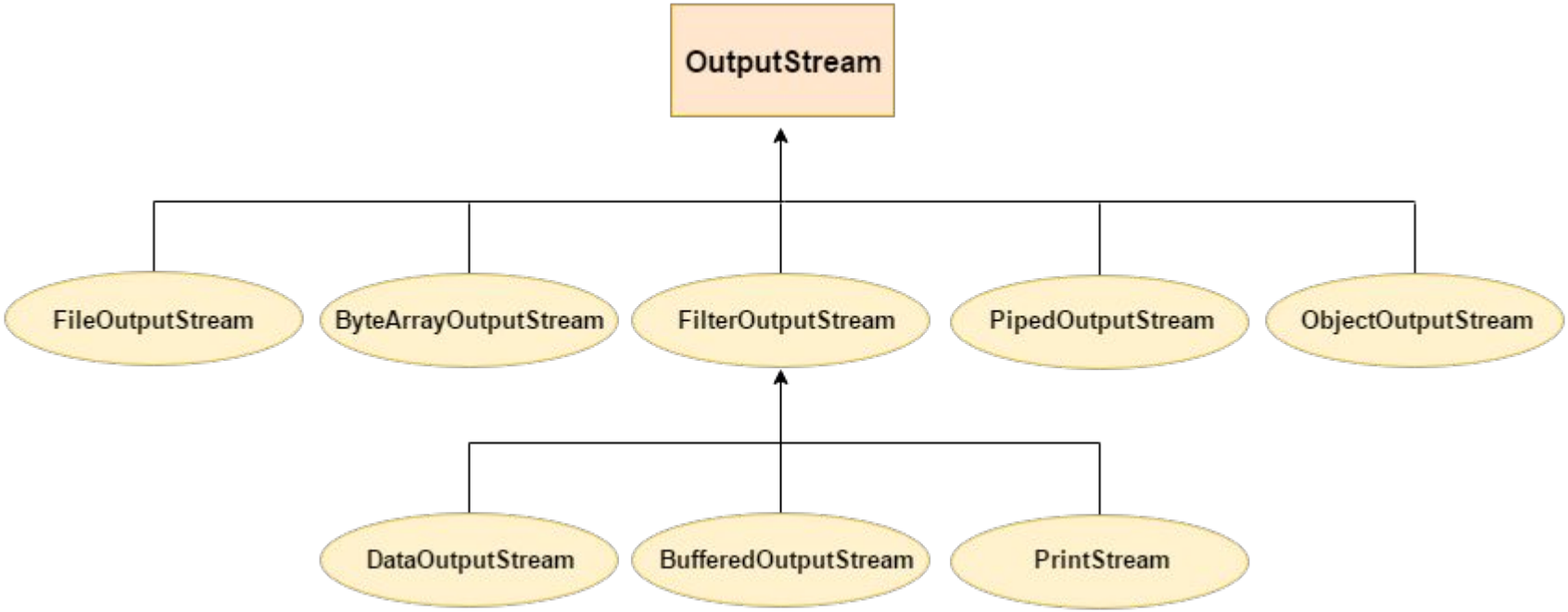
Method	Purpose
readBoolean()	Reads a boolean value.
readByte()	Reads 1 byte.
readChar()	Reads 2 bytes (character).
readDouble()	Reads 8 bytes.
readFloat()	Reads 4 bytes.
readInt()	Reads 4 bytes.
readLong()	Reads 8 bytes.
readShort()	Reads 2 bytes.
readUTF()	Reads a modified UTF-8 string.

Unicode Transformation Format – 8-bit - UTF-8 helps computers represent text from almost all languages, including English, Telugu, Hindi and many others. Size varies from 1–4 bytes.

Java Output Stream Classes

- OutputStream classes is used to write 8-bit bytes and supports a number of output-related methods
  - Writing bytes
  - Closing streams
  - Flushing streams
    - etc.

OutputStream Classes Hierarchy



Important points

- FileOutputStream → File
- ByteArrayOutputStream → Memory
- DataOutputStream → Primitive data
- BufferedOutputStream → Buffer
- ObjectOutputStream → Objects
- PrintStream → Printing data
- PipedOutputStream → Thread communication
- FilterOutputStream → Additional functionality



Some methods in output stream classes

Method	Description
`write ( )`	Write a byte from the input stream
`write (byte b[ ])`	Write all bytes in the array b to the output steam
write (byte b[ ], int n, int m)	Write m bytes from array b starting from n^{th} byte
`close( )`	Close the output stream
`flush( )`	Flushes the output stream

Example:

DataOutputStream

`writeShort( )`	`writeDouble( )`
`writeInt( )`	`writeLine( )`
`writeLong( )`	`writeChar( )`
`writeFloat( )`	`WriteBoolean( )`
`writeUTF( )`	

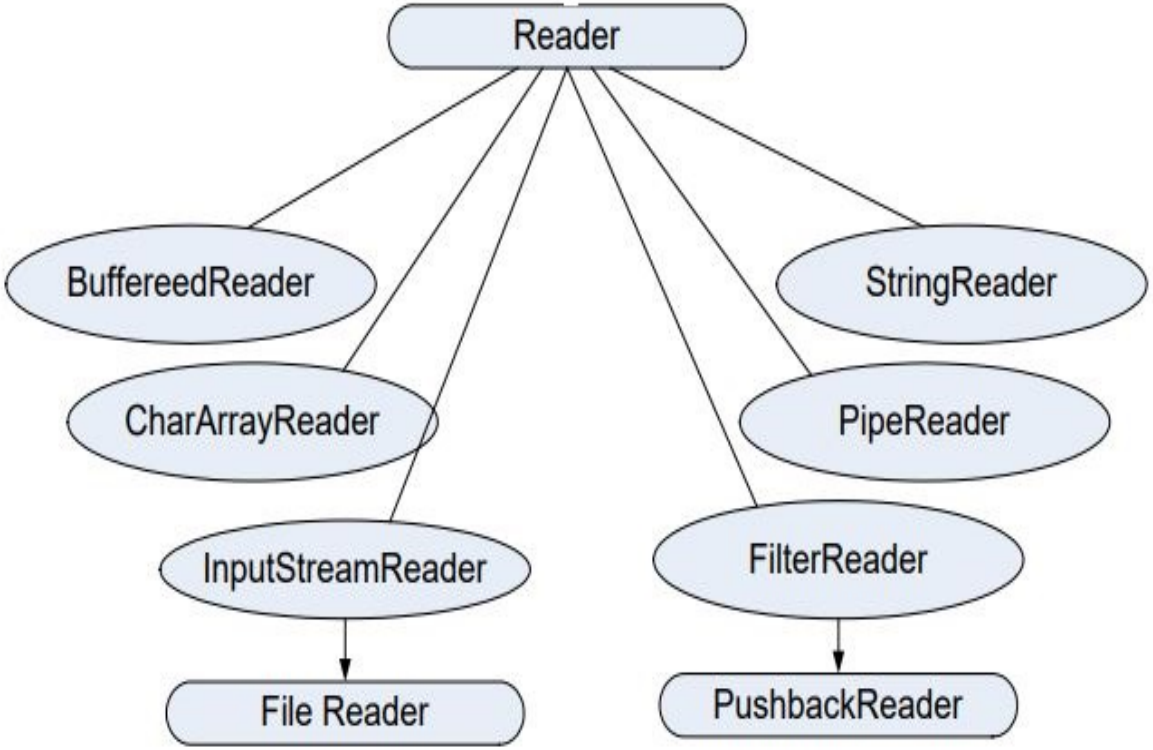
Useful Methods of OutputStream Class

Method	Description
public void write(int) throws IOException	It is used to write a byte to the current output stream.
public void write(byte[])throws IOException	It is used to write an array of byte to the current output stream.
public void flush() throws IOException	It flushes the current output stream.
public void close() throws IOException	It is used to close the current output stream.

Character stream classes

- Character stream classes is used to read and write characters and supports a number of input-output related methods
- Reader stream classes
  - To read characters from files.
  - In many way, identical to InputStream classes.
- Writer stream classes
  - To write characters into files.
  - In many way, identical to OutputStream classes.

Reader


Reader
BuffereedReader
StringReader
CharArrayReader
PipeReader
InputStreamReader
FilterReader
File Reader
PushbackReader

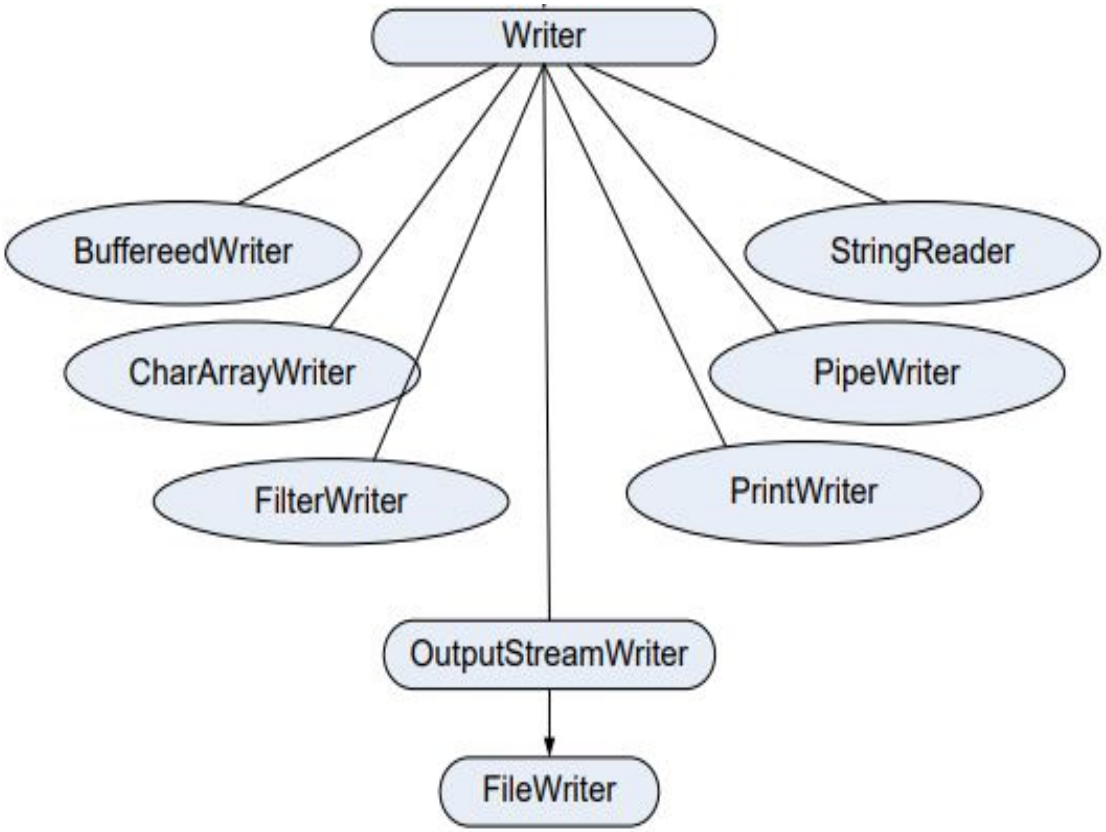
Reader class Methods

- int read() - reads one character of data from the input stream
- int read(char[] array) - reads characters from the stream and stores in the specified array
- int read(char[] array, int offset, int numChars) – Attempt to read numChars into array starting at offset.
- void mark(int numberofchars) - marks the position in the input stream up to which data has been read
- void reset() - returns the control to the point in the stream where the mark was set
- Boolean markSupported() - checks if the mark() and reset() method is supported in the stream
- long skips(long numberofchars) - skips and discards the specified number of characters from the input stream
- void close() - closes the input stream

BufferedReader class Methods

Method	Description
int read()	It is used for reading a single character.
int read(char[] cbuf, int offset, int len)	It is used for reading characters into an array
String readLine()	It is used for reading a line of text.
boolean ready()	It is used to test whether the input stream is ready to be read.
long skip(long n)	It is used for skipping the characters.
void close()	Closes the stream

Writer


Writer
BuffereedWriter
CharArrayWriter
FilterWriter
StringReader
PipeWriter
PrintWriter
OutputStreamWriter
FileWriter

Writer class Methods

- **void write(int c)** - Writes a single character (represented by an integer).
- **void write(char[] cbuf)** - Writes an array of characters to the output stream.
- **void write(char[] cbuf, int off, int len)** - Writes a portion of a character array, starting at off and writing len characters.
- **void write(String str**) - Writes a String to the output stream.
- **Writer append(CharSequence csq)** - Appends a character sequence (like a String or StringBuilder) to the writer. This is a convenient method for adding characters to the stream.
- **void flush()** - Flushes the stream, ensuring that any buffered characters are written out. This is especially useful with buffered streams like BufferedWriter
- **void close()** - Closes the stream and releases any system resources associated with it. After calling this method, no more data can be written to the stream.

File Class

- The File class is an abstract representation of a file or directory path in the filesystem.
- This class allows you to manipulate files and directories (e.g., create, delete, check properties) but doesn’t read or write file content.
- To read or write content, you need to use classes like FileInputStream, FileOutputStream, FileReader, FileWriter

Constructors:

- File(String pathname): Creates a File instance for the specified pathname
- File(String parent, String child) - Creates a File instance by combining a parent directory with a child file or directory.

```
File parent = new File("/user/docs");
File file = new File(parent, "example.txt");
```

Methods

Purpose	Method and Description
File and Directory creation	createNewFile(): atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist. mkdir() - Creates a directory mkdirs() – Creates a nested directories

Purpose	Method and Description
File properties	boolean exists() String getName() String getPath() String getAbsolutePath() int length() Date lastModified() boolean canRead() boolean canWrite() boolean canExecute() boolean isFile() boolean isDirectory() String[] list()

FileInputStream Class

- FileInputStream(File file): Creates a FileInputStream by opening a connection to the specified file.
- FileInputStream(String pathname): Creates a FileInputStream by opening a connection to the file specified by the path.

FileOutputStream class

- **FileOutputStream(File file)**: Creates a FileOutputStream that writes to the specified File.
- **FileOutputStream(String path)**: Creates a FileOutputStream that writes to the file specified by the path.
- **FileOutputStream(File file, boolean append)**: Creates a FileOutputStream for the specified file, with an option to append data instead of overwriting.
- **FileOutputStream(String path, boolean append)**: Creates a FileOutputStream for the specified path, with an option to append.